

Projeto de Segurança de Sistemas 2026.1

Segurança em Aplicações Web / Maio de 2026

Contexto

Divididos em grupos (no máximo 3 pessoas), os alunos deverão pesquisar vulnerabilidade de segurança em aplicações web, compreender sua causa, demonstrar sua ocorrência em um ambiente controlado e apresentar técnicas para prevenção e correção. A demonstração deverá ocorrer exclusivamente em laboratório local, aplicação propositalmente vulnerável ou ambiente autorizado pelo professor.

A apresentação deverá conter: conceito da vulnerabilidade, exemplo de cenário vulnerável, demonstração prática, análise de impacto, correção aplicada e checklist de boas práticas. O grupo também deverá entregar um relatório técnico e, quando aplicável, o código-fonte ou instruções para execução do laboratório.

É expressamente proibido testar, explorar ou demonstrar ataques contra sistemas reais, públicos, institucionais ou de terceiros. O objetivo da atividade é compreender os riscos para desenvolver aplicações mais seguras.

Vulnerabilidades sugeridas

1. Controle de acesso quebrado.

Usuários acessando recursos que não deveriam.

Exemplos de estudo:

- Acesso a dados de outro usuário.
- Alteração de identificadores na URL.
- Falha em permissões no backend.
- Diferença entre esconder botão no frontend e validar permissão no servidor.

Demonstração esperada:

- Mostrar uma funcionalidade vulnerável em que um usuário comum acessa dados administrativos ou de outro usuário.
- Corrigir com validação de autorização no backend.

Mitigações:

- Controle de acesso baseado em papéis ou políticas.
- Verificação de permissão em todas as rotas sensíveis.
- Testes automatizados para cenários de autorização.
- Evitar confiar apenas no frontend.

2. SQL Injection e injeções.

Entrada maliciosa sendo interpretada como comando.

Exemplos de estudo:

- SQL Injection.
- Command Injection.
- NoSQL Injection.

Demonstração esperada:

- Mostrar uma consulta vulnerável em aplicação local.
- Demonstrar que entradas não tratadas alteram o comportamento da consulta.
- Corrigir usando consultas parametrizadas ou ORM corretamente.

Mitigações:

- Prepared statements.
- Uso correto de ORM.
- Validação de entrada.
- Princípio do menor privilégio no banco.
- Evitar concatenação de strings em consultas.

3. XSS (Cross Site Scripting).

Execução de código no navegador da vítima.

Exemplos de estudo:

- XSS refletido.
- XSS armazenado.
- XSS baseado em DOM.

Demonstração esperada:

- Mostrar uma página vulnerável que renderiza a entrada do usuário sem tratamento.
- Demonstrar o impacto no ambiente local.
- Corrigir com escape de saída, sanitização e Content Security Policy.

Mitigações:

- Escape automático em templates.
- Sanitização de HTML quando necessário.
- Content Security Policy.
- Cookies com HttpOnly, Secure e SameSite.
- Evitar inserir dados do usuário diretamente no DOM.

4. Autenticação e sessão.

Problemas no login, senha, token de sessão.

Exemplos de estudo:

- Senhas fracas.
- Ausência de proteção contra força bruta.
- Tokens previsíveis.
- Sessão sem expiração.
- Cookies mal configurados.

Demonstração esperada:

- Mostrar uma aplicação com sessão mal configurada.
- Demonstrar risco de cookie inseguro ou ausência de bloqueio após tentativas.
- Corrigir com boas práticas de autenticação.

Mitigações:

- Hash de senha com algoritmos adequados.
- MFA quando aplicável.
- Rate limiting.
- Expiração de sessão.
- Cookies seguros.
- Recuperação de senha com token temporário e uso único.

5. CSRF.

Induzir o navegador autenticado da vítima a executar uma ação indesejada.

Exemplos de estudo:

- Alterar e-mail.
- Trocar senha.
- Enviar formulário sem intenção do usuário.
- Executar ação administrativa sem token CSRF.

Demonstração esperada:

- Mostrar uma ação sensível sem proteção CSRF.
- Demonstrar o risco em ambiente controlado.
- Corrigir com token CSRF e configuração adequada de cookies.

Mitigações:

- Token CSRF.
- Cookies SameSite.
- Validação de origem/referer em ações sensíveis.
- Uso adequado de métodos HTTP.
- Re-autenticação para ações críticas.

6. Upload inseguro.

Envio de arquivos maliciosos ou indevidos para o servidor.

Exemplos de estudo:

- Upload de arquivos com extensão perigosa.
- Falha na validação de MIME type.
- Armazenamento em pasta pública executável.
- Sobreescrita de arquivos.
- Arquivos grandes causando indisponibilidade.

Demonstração esperada:

- Mostrar um upload vulnerável em laboratório.
- Demonstrar riscos sem executar nada contra o ambiente real.
- Corrigir com validação, renomeação, armazenamento seguro e limitação de tamanho.

Mitigações:

- Lista permitida de extensões.
- Verificação de tipo real do arquivo.
- Renomear arquivos.
- Armazenar fora do diretório público executável.
- Limitar tamanho.
- Antivírus ou análise assíncrona.
- URLs temporárias para download.

7. Configuração insegura e exposição de informações.

Falhas causadas por ambiente mal configurado.

Exemplos de estudo:

- Debug ativo em produção.
- Erros exibindo stack trace.
- Credenciais em arquivos públicos.
- CORS permissivo demais.
- Headers de segurança ausentes.
- Serviços administrativos expostos.

Demonstração esperada:

- Mostrar uma aplicação com mensagens de erro sensíveis.
- Identificar configurações inseguras.
- Corrigir com variáveis de ambiente, headers e configuração adequada.

Mitigações:

- Desativar debug em produção.
- Configurar CORS restritivo.
- Usar *secrets* fora do código.
- Headers de segurança.
- Hardening de servidor.
- Separação entre ambiente de desenvolvimento e produção.

8. Segurança em APIs.

Segurança específica em APIs REST/GraphQL.

Exemplos de estudo:

- Falta de autenticação em endpoint.
- Exposição excessiva de dados.
- Mass assignment.
- Ausência de rate limit.
- Falha de autorização por objeto.

Demonstração esperada:

- Mostrar uma API vulnerável que retorna ou altera dados indevidos.
- Corrigir com DTOs, validação, autorização e limitação de requisições.

Mitigações:

- Autorização por endpoint e por objeto.
- DTOs de entrada e saída.
- Rate limiting.
- Validação de payload.
- Logs de auditoria.
- Não expor campos sensíveis.

Estrutura da apresentação de cada grupo

Cada grupo deve seguir o mesmo roteiro:

1. **Conceito do ataque**
Explicar o que é a vulnerabilidade.
2. **Cenário vulnerável**
Apresentar uma pequena aplicação, tela, endpoint ou fluxo com a falha.
3. **Demonstração controlada**
Mostrar a exploração somente em laboratório.
4. **Impacto**
Explicar o que poderia acontecer em um sistema real.
5. **Correção**
Mostrar código, configuração ou arquitetura corrigida.
6. **Checklist preventivo**
Listar boas práticas para evitar o problema.
7. **Referências**
Citar OWASP, documentação técnica e materiais usados.

Ambiente prático

Opção 1 – OWASP Juice Shop

É uma aplicação propositalmente insegura para treinamento e cobre vulnerabilidades do OWASP Top 10.

Opção 2 – DVWA

Aplicação clássica para demonstração de vulnerabilidades como SQL Injection, XSS, CSRF e upload inseguro.

Opção 3 – Miniaplicação criada pelo grupo

O grupo cria uma aplicação simples com:

- Uma tela vulnerável.
- Um endpoint vulnerável.
- Uma versão corrigida.
- Um README explicando como executar.

Entregáveis

Cada grupo deve entregar:

1. Slides da apresentação.
2. Relatório técnico curto, com 4 a 8 páginas com:
 - a. Introdução
 - b. Descrição da vulnerabilidade
 - c. Cenário vulnerável
 - d. Demonstração em ambiente controlado
 - e. Impactos possíveis
 - f. Correção aplicada
 - g. Boas práticas preventivas
 - h. Conclusão
 - i. Referências
3. Código-fonte ou ambiente Docker, quando aplicável.
4. README de execução do laboratório.
5. Checklist de mitigação.
6. Referências utilizadas.

Regras de segurança da atividade

1. As demonstrações só podem ocorrer em ambientes locais, máquinas virtuais, containers ou aplicações propositalmente vulneráveis.
2. É proibido testar ataques em sites, sistemas públicos, sistemas da instituição ou aplicações de terceiros.
3. Os alunos devem demonstrar o ataque de forma didática, sem disponibilizar scripts ofensivos reutilizáveis contra ambientes reais.
4. Toda apresentação deve conter obrigatoriamente a parte defensiva: prevenção, correção, validação e boas práticas.
5. O relatório deve explicar o risco, o impacto e a mitigação, não apenas “como explorar”.

Observações

- Testar em sistema real sem autorização: nota zero na atividade.
- Apresentar apenas ataque sem mitigação: perda significativa de pontos.
- Copiar conteúdo sem explicar: nota zero na atividade.
- Não demonstrar ambiente controlado: perda de pontos.